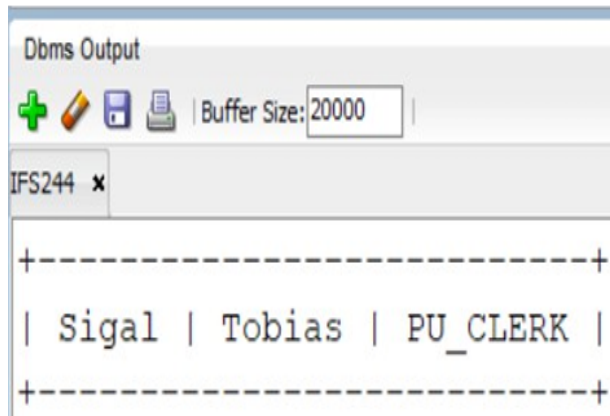


### Question 1

Write a PL/SQL routine in SQL Developer that will solicit an **employee ID**, and **declare variables** to hold an employee **first name**, **last name**, and **job ID**. Run a query that **selects** from the **EMPLOYEES table** using the **user-input ID** to **get data for the variables**. The routine must then print the extracted data in the following format:



The purpose of this exercise is for you to **create a program that will adjust itself to the different names and jobs produced by different employee IDs**, and **print them correctly inside a row as above**, with the **correct number of n-dashes for each set of values**.

Note **that there is a space char before and after each string in the row**. In PL/SQL, to print just a single character to the output buffer, you must call `DBMS_OUTPUT.PUT` followed by `DBMS_OUTPUT.NEW_LINE` to flush the buffer and print the characters – see an example below:

```
BEGIN
  FOR i IN 1..5 LOOP
    DBMS_OUTPUT.PUT('X');
  END LOOP;
  DBMS_OUTPUT.NEW_LINE;
END;
```

## Question 2

WRITE a PL/SQL block in Oracle SQL Developer using the HR schema to query and display all employees in department 60 (IT), showing each employee's full name, job title, and monthly salary by declaring scalar variables with the %TYPE attribute, explicitly joining the HR.EMPLOYEES and HR.JOBS tables, and fetching data through an explicitly declared cursor using a basic loop (LOOP ... EXIT WHEN ... END LOOP) without a cursor FOR loop, while utilizing an inner IF statement to append "(High Earner)" if their salary exceeds 5000 or "(Standard Earner)" otherwise.

```
Alexander Hunold works as Programmer with salary 9000 (High Earner)
Bruce Ernst works as Programmer with salary 6000 (High Earner)
David Austin works as Programmer with salary 4800 (Standard Earner)
Valli Pataballa works as Programmer with salary 4800 (Standard Earner)
Diana Lorentz works as Programmer with salary 4200 (Standard Earner)
```

```
PL/SQL procedure successfully completed.
```

### Question 3

Create a procedure named `dept_salary_report` that takes one parameter, a department ID. In this procedure, you should print the department name along with its average salary. After this, use a cursor to loop through each employee in that department and print out their first\_name, job\_id and salary. Ensure that each row is tagged as either "Above average" or "Below average" based on if it is above or below the departments average salary. If the department does not exist, make sure to handle that by catching/trapping the exception and including a suitable message.

Once the procedure is created, make sure to call/invoke the procedure with department ID 60 and the report formatting match the below output with the dashed borders. The borders length is 60.

The goal of this question is to test your use of built-in functions, cursors, loops and following a given format.

#### DO NOT:

- Hard Code currency values. Use your built-in functions to format salary values.

```
Department: IT
Average Salary: R5,760.00
```

```
-----
Alexander Hunold      IT_PROG      R9,000      (Above Avg)
Bruce Ernst           IT_PROG      R6,000      (Above Avg)
David Austin          IT_PROG      R4,800      (Below Avg)
Valli Pataballa       IT_PROG      R4,800      (Below Avg)
Diana Lorentz         IT_PROG      R4,200      (Below Avg)
-----
```

```
No department found with ID 60000.
```